

Hardware-Deterministic Modulo-2 Polynomial Division & Telemetry Error Detection Engine

PROBLEM STATEMENT

Silent Telemetry Corruption & Noise

- **Context:** Industrial telemetry logs sent across RS-485 / CAN / Modbus are vulnerable to electromagnetic interference (EMI) and bit decay.
- **Impact:** Flipping a single bit alters sensor data (e.g. TEMP:82.4C → TEMP:92.4C), causing false emergency shutdowns or missed faults.
- **Solution:** A pure handwritten Modulo-2 CRC-32 engine with a session-fixed 33-bit divisor providing 100% burst error detection up to 32 bits.

Repo: <https://github.com/rogeralukka/OS-CN-Product-8>

TEAM MEMBER RESPONSIBILITIES (GUIDELINE SECTION 10)

Group VIII Demonstration Roles

- 1. ROGER A RAJU (Lead / Architecture):** Problem statement, product overview, software workflow & dataset management (Slides 1 & 2).
- 2. K. ROHIT GOUD (Algorithms Lead):** GF(2) Galois field mathematics, Modulo-2 XOR logic & CRC-32 division (Slides 3 & 4).
- 3. G. SIDDHARTHA (Data Ingestion Lead):** ASCII-to-binary mapping, file input parsing (.log/.txt) & dashboard integration (Slides 4 & 5).
- 4. P. HARSHAVARDHAN (Backend & Telemetry):** Flask REST API (/api/compute, /api/verify), session divisor & live step telemetry (Slides 5 & 6).
- 5. G. REVANTH (QA & Verification Lead):** Test cases (TC-01..TC-05), error simulation, challenges & summary (Slides 7 & 8).

Introduction to the Product & Software Workflow

PRODUCT SUMMARY

SLIDE 02 / 10

Area	What is Implemented in Product 8
Problem Requirement	Detects bit-level telemetry mutations, transmission noise, and log tampering in real-time industrial SCADA systems. CRC is required because additive checksums fail on bit swaps and burst noise.
Implementation Plan	1. User Input (Text/File) → 2. ASCII-to-Binary Conversion → 3. Append 32 Zeros → 4. Handwritten Modulo-2 Division (xordiv) → 5. 32-bit CRC Generated → 6. Codeword Re-division at Receiver → 7. Syndrome Zero Check.
Software & Stack Used	Backend: Python 3, Flask REST API, Flask-CORS. Frontend: Vanilla HTML5, CSS3 (Warm Monotone Single-Viewport), JavaScript Fetch API. Tools: VS Code, Git, Chrome.
Input / Test Data	Supports direct binary streams, raw log text messages, and full industrial .log / .txt files (10 pre-built demo reports provided in Downloads).
Final Output	Calculated 32-bit CRC, Hex digest (0x4964A71B), complete bitwise step-by-step audit trail, and instant binary ACCEPT (0 ³²) or REJECT chip.

Operating Principle of CRC: Modulo-2 Division

DISCRETE MATH

SLIDE 03 / 10

1. POLYNOMIAL MAPPING

Galois Field GF(2)

- Bitstream is treated as polynomial coefficients over GF(2).
- Example: 'A' = $01000001_2 \rightarrow (P(x) = x^6 + 1)$.
- **Arithmetic Rule:** Addition and subtraction are identical to bitwise XOR ($\oplus$).
- **No Carries/Borrows:** Enables line-rate single-cycle hardware execution.

2. DIVISION & PADDING

32-Zero Appending

- Sender appends 32 zeros: $(M(x) \cdot x^{32})$.
- Long division performed with 33-bit Generator $(G(x))$.
- If leading bit is 1, XOR with divisor; if 0, shift 1 bit right.
- $(M(x) \cdot x^{32} = Q(x)G(x) \oplus R(x))$. Remainder $(R(x))$ is the 32-bit CRC.

3. SYNDROME PROOF

Receiver Zero Remainder

- Codeword Transmitted: $(T(x) = M(x) \cdot x^{32} \oplus R(x))$.
- $(T(x) = Q(x)G(x) \oplus R(x) \oplus R(x) = Q(x)G(x))$.
- Receiver calculates: $(T'(x) \pmod{G(x)} = R'(x))$.
- **Decision Rule:** If $(R'(x) == 0^{32}) \rightarrow$ ACCEPT (No Error); else REJECT.

Python Implementation of CRC (crc_checker.py)

SOURCE CODE

SLIDE 04 / 10

HANDWRITTEN PYTHON LOGIC

Core Functions (No zlib)

```
def ascii2binary(text):
    return ''.join(f'{ord(c):08b}' for c in text)

def xordiv(dividend, divisor):
    d, m = list(dividend), len(divisor) # 33 bits
    for i in range(len(d) - m + 1):
        if d[i] == '1':
            for j in range(m):
                d[i+j] = '1' if d[i+j] != divisor[j]
    else '0'
    return ''.join(d[-(m-1):]) # Last 32 bits

def compute_crc(text):
    return xordiv(ascii2binary(text) + '0'*32, get_divisor())
```

Input Type	Processing Flow
Binary Input	Binary String → Direct Validation → xordiv() → CRC Result
Text Input	Text → ascii2binary() → Append 32 Zeros → xordiv() → CRC
File Input	File Upload → FileReader → UTF-8 text → ascii2binary() → CRC

FUNCTION SPECIFICATION

Data Invariant

- **Input:** UTF-8 log message string or file buffer.
- **Processing:** Modulo-2 division against session divisor.
- **Output:** 32-bit CRC, Hex digest, full step trace.
- **Shared Divisor:** compute_crc() and verify() both call get_divisor().

Dashboard & Application Details

Aspect	System Details & Implementation
Software Used	Flask 2.3+ REST backend (Port 5000), Vanilla JS frontend, Modern HTML5/CSS3 with Warm Monotone palette.
Intended Users	Industrial plant operators, SCADA instrumentation engineers, edge gateway monitors, QA compliance auditors.
Dashboard Inputs	Original log textarea, file upload (.txt / .log), random telemetry generator, received log textarea, received CRC input.
Dashboard Outputs	Calculated 32-bit CRC, Hex digest, step-by-step binary/padding/remainder calculations, ACCEPT/REJECT status badge.

DASHBOARD-TO-CODE PIPELINE

System Execution Flow

User Input (Text/File Upload) → Dashboard State → POST /api/compute → `ascii2binary()` → `xordiv()` → Return CRC & Steps JSON → Live Dashboard Telemetry Rendering → Receiver Re-ingestion → POST /api/verify → Result Chip [✓ ACCEPT / ✗ REJECT]

Testing & Test Cases Matrix

Test ID	Input Data	Operation	Expected Result	Actual Result	Status
TC-01	Binary: 101100101	CRC-32 Modulo-2	32-bit CRC generated	Correct CRC calculated	PASS
TC-02	Text: [TURBINE_01] RPM: 3450	CRC Computation	32-bit CRC + Hex returned	0x367B4724 displayed	PASS
TC-03	File: Report_01_Turbine.log	File Upload CRC	Complete file CRC computed	Steps 1-5 displayed	PASS
TC-04	Modified: RPM: 3450 → RPM: 3451	Error Detection	Non-zero Remainder → REJECT	Remainder != 0 (REJECT)	PASS
TC-05	Unmodified: Valid Log + CRC	Verification Check	Remainder = 0 ³² → ACCEPT	Remainder = 0 (ACCEPT)	PASS

Mandatory Test Requirements Verified:

✓ Valid binary bitstream input tested | ✓ UTF-8 string telemetry tested | ✓ Real industrial .log/.txt files tested | ✓ Single-bit flipped error detected (REJECT) | ✓ Unmodified original log verified authentic (ACCEPT)

Errors and Challenges Faced

Problem / Challenge	Root Cause	Engineering Solution	Learning Outcome
Polynomial Bit Length	Using 32-bit divisor for 32-bit CRC caused degree underflow.	Generated 33-bit divisor with MSB=1 and LSB=1 (degree 32).	Understood that degree-n polynomial requires n+1 coefficients.
Modulo-2 Long Division	Integer division propagates carries, producing arithmetic quotient.	Built handwritten bitwise XOR loop shifting without carries/borrows.	Mastered Galois Field GF(2) long division algorithms.
Divisor Synchronization	Sender & receiver generated different random polynomials.	Created singleton <code>get_divisor()</code> fixed for entire session in memory.	Learned session-level state management and protocol parity.
Single-Screen Layout	Long step outputs created ugly outer window scrolling.	Engineered 2-row CSS grid with internal 6px custom scrollbars.	Mastered responsive viewport budgeting without layout thrashing.

Additional Features & Live Demonstration Sequence

LIVE
DEMONSTRATIONSLIDE 08 /
10

WORKING ADDITIONAL FEATURES

System Highlights

- **Session-Fixed 33-Bit Generator:** Singleton polynomial synchronized via GET /api/divisor.
- **10-Report Demo Pack:** Industrial SCADA test reports pre-populated in Downloads for file upload testing.
- **Step-by-Step Telemetry:** Live ASCII binary, 32-0 padding, divisor, and remainder inspectability.
- **Dual-Row Single Viewport UI:** 2-row layout with subtle 6px scrollbars and instant random log generator.

DEMONSTRATION PLAYBOOK

5-Step Live Sequence (Guideline Sec 12)

1. **PPT Window:** Explain Problem, Math & Architecture (Slides 1-4).
2. **Dashboard Window:** Upload Report_01_Turbine.log & compute CRC.
3. **Code Window:** Trace `ascii2binary()` and `xordiv()` in `crc_checker.py`.
4. **Dashboard Window:** Demonstrate Accept (0^{32}) & Reject (modify 1 character).
5. **Summary:** Conclude with integrity guarantees & Viva defense.

Three Windows Prepared for Live Demo: 1. PowerPoint Presentation | 2. Dashboard (<http://localhost:5000>) | 3. Python Code (VS Code)

Viva Voce & Technical Defense Preparation

VIVA Q&A

SLIDE 09 / 10

MATHEMATICAL CONCEPTS

Concept Defense

- **Q: What is CRC & why used?**

Cyclic Redundancy Check; detects burst errors using polynomial long division over GF(2).

- **Q: Why 33-bit divisor for 32-bit CRC?**

A degree-32 polynomial has 33 coefficients. Dividing by degree-32 strictly leaves a remainder of degree ≤ 31 (32 bits).

- **Q: Why is XOR used instead of subtraction?**

In GF(2), addition and subtraction are identical to bitwise XOR with no carries or borrows.

- **Q: Can CRC correct errors?**

CRC is strictly for error detection. Error correction requires FEC (e.g. Reed-Solomon).

IMPLEMENTATION & VERIFICATION

Code & Verification Defense

- **Q: How is text converted before CRC?**

Each character is converted to 8-bit ASCII binary via `ascii2binary()` and concatenated.

- **Q: How does the receiver detect an error?**

Codeword $T(x) = \text{ext}\{\text{Data}\} + \text{ext}\{\text{Received CRC}\}$ is divided by $G(x)$. If remainder is $0^{32} \rightarrow \text{ACCEPT}$; else REJECT.

- **Q: What happens if one bit is changed?**

The polynomial $M'(x)$ differs from $M(x)$, resulting in a non-zero syndrome remainder $R'(x) \neq 0$.

- **Q: What error types are 100% detected?**

All single-bit errors, all double-bit errors, and all burst errors ≤ 32 bits.

Product Summary & Key Learning Outcomes

PRODUCTION READY

SLIDE 10 / 10

PRODUCT SUMMARY

System Deliverables

- **Developed Product:** Industrial Machine Log Integrity Checker (Product 8).
- **Supported Input Types:** Binary streams, UTF-8 strings, industrial .log / .txt files.
- **Supported CRC Algorithm:** Pure Modulo-2 XOR CRC-32 with session-fixed 33-bit generator polynomial.
- **Error Detection Capabilities:** 100% single bit, 100% double bit, 100% bursts ≤ 32 bits, 99.99999997% longer bursts.
- **Software Stack:** Python 3, Flask REST API, Vanilla HTML5/CSS3/JS, Warm Monotone Single-Screen UI.

LEARNING OUTCOMES

OS & Computer Networks

1. **Protocol Parity:** Implemented synchronized sender-receiver telemetry pipelines.
2. **Bitwise Computing:** Built low-level bit shifting and XOR operations without black-box libraries (no `zlib`).
3. **Error Detection:** Understood why polynomial division surpasses additive checksums.
4. **Full-Stack Integration:** Integrated Flask REST endpoints with real-time UI telemetry and FileReader APIs.
5. **Team Collaboration:** Successfully bifurcated responsibilities across 5 group members.